数值线性代数　第 13 讲：浮点运算

Lloyd N. Trefethen、David Bau III　中文阅读版

计算机发明后不久，人们就对数字设备表示实数的正确方式形成了共识. 秘密就在于浮点运算：它是科学记数法在硬件中的对应物. 在开始研究数值线性代数算法的准确性之前，必须先考察这一主题.

数字表示的局限

由于数字计算机使用有限个比特表示实数，它们只能表示实数（或者复数；本讲末尾会讨论复数）的一个有限子集. 这一局限带来两个困难. 第一，被表示的数不可能任意大或任意小. 第二，相邻表示数之间必然存在间隔.

现代计算机能够表示足够大和足够小的数，因此第一个限制很少造成困难. 例如，广泛使用的 IEEE 双精度运算允许表示最大约为 1.79×10^{308}、最小约为 2.23×10^{-308} 的数；对本书讨论的大多数问题而言，这个范围已经足够大. 换言之，溢出和下溢通常不是严重风险（但如果有人要求你计算行列式，就要特别小心）.

相反，表示数之间的间隔在科学计算中始终值得关注. 例如，在 IEEE 双精度运算中，区间 $[1,2]$ 表示为离散子集

$$1, \quad 1+2^{-52}, \quad 1+2\cdot 2^{-52}, \quad 1+3\cdot 2^{-52}, \quad \ldots, \quad 2. \tag{13.1}$$

区间 $[2,4]$ 则由同一组数乘以 2 得到：

$$2, \quad 2+2^{-51}, \quad 2+2\cdot 2^{-51}, \quad 2+3\cdot 2^{-51}, \quad \ldots, \quad 4.$$

一般地，区间 $[2^j, 2^{j+1}]$ 由式 (13.1) 中的数乘以 2^j 得到. 因此，在 IEEE 双精度运算中，相邻表示数之间的间隔从相对意义上看从不会大于

$$2^{-52} \approx 2.22 \times 10^{-16}.$$

如果使用稳定算法，这个量对大多数目的而言确实小得可以忽略（见下一讲）. 但令人意外的是，有多少粗心设计的算法最终会表现出不稳定性!

浮点数

IEEE 运算是以浮点表示实数的一种运算系统. 如今，通用计算机普遍采用这种做法. 在浮点数系统中，小数点（或二进制点）的位置与数字本身分开存储；相邻表示数之间的间隔会随数的大小成比例变化. 这与定点表示不同，后者的间隔始终相同.

具体地，考虑如下理想化的浮点数系统. 令 $\mathbb{F}$ 是实数 $\mathbb{R}$ 的一个离散子集，它由整数 $\beta \geq 2$ 和整数 $t \geq 1$ 决定；β 称为基数或底（通常取 2），t 称为精度（IEEE 单精度和双精度分别对应 24 和 53）. $\mathbb{F}$ 中包含数字 0，以及所有形如

$$x = \pm \left(\frac{m}{\beta^t} \right) \beta^e \tag{13.2}$$

的数，其中 m 是满足 $1 \leq m \leq \beta^t$ 的整数，e 是任意整数.

等价地，也可以限制

$$\beta^{t-1} \leq m \leq \beta^t - 1,$$

这样 m 的选择就是唯一的. 此时 $\pm(m/\beta^t)$ 称为 x 的小数部分或尾数，e 称为指数.

上述浮点数系统是理想化的，因为它忽略了上溢和下溢. 因此，$\mathbb{F}$ 是一个可数无穷集，并且具有自相似性：

$$\mathbb{F} = \beta\mathbb{F}.$$

机器 epsilon

$\mathbb{F}$ 的分辨率通常用一个称为机器 epsilon（machine epsilon）的数来概括. 暂时定义

$$\varepsilon_{\text{machine}} = \frac{1}{2}\beta^{1-t}. \tag{13.3}$$

（在式 (13.7) 之后我们会修改这个定义.）这个数是 1 与下一个更大的浮点数之间距离的一半. 从相对意义上看，浮点数之间的间隔最大也就是这个量. 也就是说，$\varepsilon_{\text{machine}}$ 具有如下性质：

$$\text{对所有} x \in \mathbb{R},\ \text{存在} x' \in \mathbb{F}, \quad |x - x'| \leq \varepsilon_{\text{machine}}|x|. \tag{13.4}$$

对于各种计算机上常见的 β 和 t，$\varepsilon_{\text{machine}}$ 通常位于 10^{-6} 与 10^{-35} 之间. 在 IEEE 单精度和双精度运算中，分别规定

$$\varepsilon_{\text{machine}} = 2^{-24} \approx 5.96 \times 10^{-8}, \qquad \varepsilon_{\text{machine}} = 2^{-53} \approx 1.11 \times 10^{-16}.$$

令 $\text{fl} : \mathbb{R} \to \mathbb{F}$ 表示把实数取为浮点系统中最近浮点数的函数，即舍入后的等价值（平局情形可以任意处理；如何处理平局以避免统计偏差，本身也是一个有趣的问题）. 式 (13.4) 可以用 fl 表述为：

$$\text{对所有} x \in \mathbb{R},\ \text{存在} \delta,\ |\delta| \leq \varepsilon_{\text{machine}}, \quad \text{fl}(x) = x(1+\delta). \tag{13.5}$$

也就是说，实数与其最近浮点近似之间的差，从相对意义上看总小于 $\varepsilon_{\text{machine}}$.

浮点运算

当然，仅仅表示实数是不够的；还必须对它们进行计算. 在计算机上，所有数学计算都归结为若干基本算术运算，经典的运算集合是 $+$、$-$、$\times$ 和 $\div$. 从数学上说，这些符号表示 $\mathbb{R}$ 上的运算. 在计算机上，它们有作用于 $\mathbb{F}$ 的对应运算. 通常把这些浮点运算记为 $\oplus$、$\ominus$、$\otimes$ 和 $\oslash$.

计算机可以按照下面的设计原则构造. 令 $x, y \in \mathbb{F}$ 为任意浮点数. 令 $*$ 为 $+$、$-$、$\times$ 或 $\div$ 中的一个运算，并令 $\circledast$ 为它对应的浮点运算. 那么要求

$$x \circledast y = \mathrm{fl}(x * y). \tag{13.6}$$

如果这一性质成立，那么由式 (13.5) 和 (13.6) 可以得到一个简单而有力的性质.

浮点运算基本公理
对所有 $x, y \in \mathbb{F}$，存在 δ 满足 $|\delta| \le \varepsilon_{\text{machine}}$，使得

$$x \circledast y = (x * y)(1 + \delta). \tag{13.7}$$

换言之，每一次浮点运算都精确到一个至多为 $\varepsilon_{\text{machine}}$ 的相对误差.

再次讨论机器 epsilon

本书的舍入误差分析建立在式 (13.5) 和 (13.7) 上，而不是建立在前面描述的浮点运算其他细节上. 这意味着，我们可以宽容地考虑一些并不完全按照式 (13.6) 完美执行浮点计算的硬件实现. 对于这样的机器，只要把 $\varepsilon_{\text{machine}}$ 换成一个稍大的值，式 (13.5) 和 (13.7) 仍可能成立. 例如，在中间量采用截断而不是舍入的计算机上，式 (13.7) 可能需要把 $\varepsilon_{\text{machine}}$ 换成 $2\varepsilon_{\text{machine}}$.

处理这类复杂情况的最简单方式，是保留式 (13.5) 和 (13.7) 的形式，但修改 $\varepsilon_{\text{machine}}$ 的定义. 从现在开始，假设 $\varepsilon_{\text{machine}}$ 定义为使式 (13.5) 和 (13.7) 成立的最小数. 对于包括所有采用 IEEE 运算的计算机在内的大多数计算机，这一修改不会显著改变其数值.

有时，为使式 (13.7) 成立，可能需要一个出乎意料地大的 $\varepsilon_{\text{machine}}$.1994 年底，Intel Pentium™ 微处理器曾因 IEEE 双精度标准实现中的查表错误而声名狼藉；其有效精度粗糙了 11 个数量级，$\varepsilon_{\text{machine}} \approx 6.1 \times 10^{-5}$（这个错误很快得到纠正）. 事实上，有些机器上式 (13.7) 只有在 $\varepsilon_{\text{machine}} = 1$ 时才成立. 例如，直到 20 世纪 90 年代中期，Cray 计算机上的浮点减法都具有这一性质，因为减法实现时没有“保护数字”(guard digit). 这些计算机并非毫无用处，但它们要求采用不同于本书的误差分析方式.

幸运的是，式 (13.7) 公理的优点，以及采用统一计算机算术标准的好处，近年来已经被计算机制造商广泛接受. 市场上不满足“小 $\varepsilon_{\text{machine}}$”版本式 (13.7) 的机器正在减少. 事实上，IEEE 运算本身正在迅速成为各种规模计算机的标准；截至 1996 年，它已经用于所有 IBM 兼容个人计算机，以及 SUN、DEC、Hewlett–Packard 和 IBM 制造的所有工作站.

复数浮点运算

浮点复数通常表示为一对浮点实数，作用于它们的基本运算则归结为对实部和虚部进行运算. 结果是，式 (13.7) 对复浮点数同样成立，但乘法和除法例外：对于 $\otimes$ 和 $\oslash$，必须把式 (13.3) 中的 $\varepsilon_{\text{machine}}$ 分别放大到原来量级的

$$2^{3/2} \quad \text{和} \quad 2^{5/2}.$$

一旦按这种方式调整 $\varepsilon_{\text{machine}}$，复数的舍入误差分析就可以像实数一样进行.

习题

13.1 在一对相邻的非零 IEEE 单精度实数之间，有多少个 IEEE 双精度数?

13.2 式 (13.2) 定义的浮点数系统 $\mathbb{F}$ 包含许多整数，但并不包含全部整数.

(a) 给出不属于 $\mathbb{F}$ 的最小正整数 n 的精确公式.

(b) 对 IEEE 单精度和双精度运算，n 的值分别是多少?

(c) 设法在自己的计算机上验证这一结果. 具体地，设计并运行一个程序，给出证据说明 $n-3$、$n-2$ 和 $n-1$ 属于 $\mathbb{F}$，而 n 不属于 $\mathbb{F}$. $n+1$、$n+2$ 和 $n+3$ 呢?

13.3 考虑多项式

$$\begin{aligned} p(x) &= (x-2)^9 \\ &= x^9 - 18x^8 + 144x^7 - 672x^6 + 2016x^5 \\ &\quad - 4032x^4 + 5376x^3 - 4608x^2 + 2304x - 512. \end{aligned}$$

(a) 对 $x = -1.920, -1.919, -1.918, \ldots, 2.080$ 作图；通过系数 $1, -18, 144, \ldots$ 计算 $p(x)$.

(b) 再作一次同样的图，但这次使用表达式 $(x-2)^9$ 计算 $p(x)$.

13.4 多项式

$$p(x) = x^5 - 2x^4 - 3x^3 + 3x^2 - 2x - 1$$

有三个实零点. 对 p 使用初值 $x_0 = 0$ 的 Newton 方法，会产生快速收敛到某个零点 $x_* \approx -0.315$ 的估计序列 $x_1, x_2, x_3, \ldots$.

(a) 在 $\varepsilon_{\text{machine}} \approx 10^{-16}$ 的浮点运算中计算 $x_1, \ldots, x_6$. 估计这些数中每一个有多少位数字正确?

(b) 借助 MAPLE 或 MATHEMATICA 等符号代数系统，再次精确计算 $x_1, \ldots, x_6$. 每个 x_j 都是有理数. 对于每个 j，分子和分母各有多少位数字?